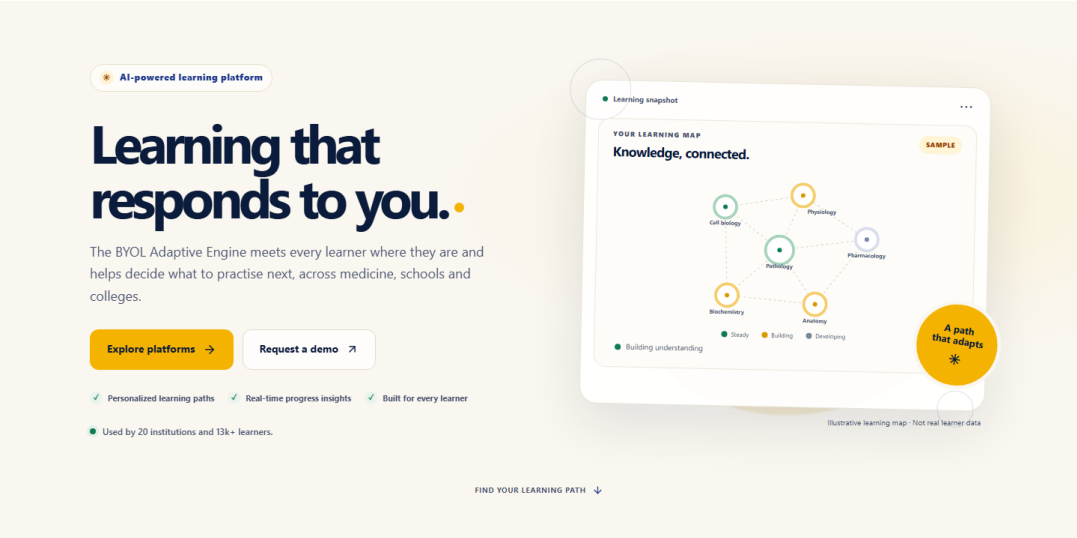
BYOL ACADEMY

# Complete codebase, file by file

A reference that explains what each authored project file does, how the important code works, and how pages, components, content, tests and deployment checks connect.




*We will trace this rendered page back to its route, component, content and style files.*

> Scope
> This guide covers all source files under src/, the unit and browser tests, root project configuration, documentation, and the tagged GitHub Actions CI workflow. It does not reproduce every line of third-party libraries.

# How to read this file-by-file guide

Each file entry answers three questions: why the file exists, what its key code is responsible for, and which other files it works with. Small excerpts show the shape of the code; the actual source remains the authority.

| Area | File pattern | Purpose |
|---|---|---|
| Routes/pages | src/app/**/page.tsx | Turn URLs into visible pages. |
| Shared interface | src/components/*.tsx | Reusable UI and interactions. |
| Text/sample data | src/content/*.ts | Content records used by pages/components. |
| Reusable logic | src/lib/**/*.ts | Calculations, validation, analytics and SEO. |
| Presentation | src/styles/globals.css | Shared tokens and CSS styling. |
| Verification | tests/**/*.test.ts, tests/e2e/*.ts | Tests for logic, routes and browser behavior. |
| Project setup | Root config, package files, .github/** | Tell tools how to compile, test and automate. |

### The connection map

```
Visitor URL -> Next.js route -> React page/component
            |                   |
            |                   +--> content data
            |                   +--> helper logic
            |                   +--> global CSS classes
            v
            HTML response -> browser draws and handles interaction

UI form -> POST /api/lead -> validate -> rate limit -> email provider
```

File references below are written as code paths. In VS Code use Ctrl+P and type the file name. The source tree does not include node_modules or .next as authored code; those are generated/downloaded folders.

# Part I - Application routes and pages

Next.js App Router maps folders and special files to URLs. Most page components here are server-rendered by default; they import only the interactive components that need browser state.

### src/app/layout.tsx

The root shared layout wraps every route. It imports the global stylesheet, header/footer, analytics wrapper and brand data.

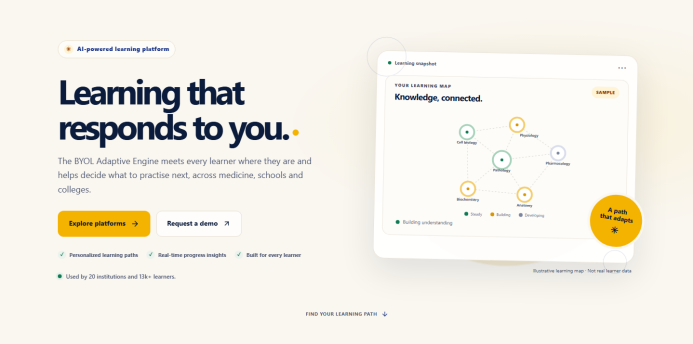
The exported metadata gives the whole site its default title/description, canonical URL base and social-sharing values. Preview deployments get no-index metadata. RootLayout emits the document language, skip link and semantic main element. The `{children}` slot is where the selected page is inserted. Vercel Analytics is rendered only when the public analytics flag is true.

```
export default function RootLayout({ children }) {
  return (
    <html lang="en-IN">
      <body>
        <a href="#main">Skip to content</a>
        <Header />
        <main id="main">{children}</main>
        <Footer />
        <AnalyticsEvents />
        {analyticsEnabled && <Analytics />}
      </body>
    </html>
  );
}
```

### src/app/page.tsx

This is the homepage route, `/`. It composes the page sections and imports content, icons, reusable visual components and SEO structured-data objects.

The first lines choose icon components for product/audience IDs. HomePage filters approved platforms and statistics, then returns ordered semantic sections: hero, audience choices, product cards, benefits, conceptual engine steps, demo, proof, responsible AI, institution CTA and closing CTA. Arrays are rendered with `.map()` and stable `key` values. Internal links use Next Link; external platform destinations use anchors with safe rel attributes. JsonLd adds search metadata; MasteryMap is used as a visual.




*Homepage hero from the page.tsx route; many sections follow it in the same file.*

## src/app/about/page.tsx

The `/about` page tells BYOL's story, describes the conceptual learning loop, summarizes research considerations and lists design principles.

It exports route-specific metadata. Its default page function is server-rendered and composes sections from `copy.about`, `engineSteps` and `principles`. Links to `/#platforms`, `/#how-it-works` and `#research` point to the relevant route/section. Its copy explicitly distinguishes conceptual design from a claim that a model is deployed.

## src/app/blog/page.tsx

The `/blog` route is a deliberately honest empty state because no approved articles/CMS feed are present.

Metadata gives the route a title and description. The page renders the copy from `copy.blog` and links visitors to the research section. It avoids generating fake posts.

## src/app/contact/page.tsx

The `/contact` page displays contact context and the lead form component.

Its props include `searchParams`, which in this Next setup is a Promise; the async component awaits it. `?intent=institution` chooses a different description and passes intent to LeadForm. The platform list is reused for external product links. The footnote points to the pending Privacy page.

```
const params = await searchParams;
const intent = params.intent;
<LeadForm initialIntent={intent} />
```

## src/app/privacy/page.tsx

The `/privacy` route is a noindex placeholder, not a privacy policy.

It exports route-specific metadata and displays the pending legal-review state. The visible notice states that the page is not a Privacy Policy and that lead capture remains disabled until legal text and consent language are approved. This prevents placeholder text being mistaken for legal advice.

## src/app/terms/page.tsx

The `/terms` route is a noindex placeholder until BYOL supplies approved Terms of Use.

It follows the same pattern as Privacy: metadata, clear pending status, statement that this is not a legal terms document, and a contact link.

## src/app/styleguide/page.tsx

The `/styleguide` route previews design tokens and reusable controls for internal review.

It has noindex/no-follow metadata, displays color swatches, button styles, type samples and content-approval status. The swatches are created from a short data array, demonstrating that repeated interface items can be data-driven.

## src/app/not-found.tsx

Next.js renders this component for a route that does not exist.

It presents a branded 404 message and two recovery links: home and platforms. It uses Next Link for local navigation and icons marked aria-hidden because they are decorative.

## src/app/error.tsx

This client error boundary gives visitors a retry action if a route fails while rendering.

The `error` object and `reset` function are supplied by Next.js. An effect logs the digest or error name, not submitted user data. Calling reset retries the segment; the link returns home. The `use client` marker is needed for the effect and button callback.

### src/app/opengraph-image.tsx

This special route generates a 1200 by 630 PNG used as a social-sharing card.

Next.js ImageResponse renders JSX-like layout into an image. `size`, `contentType` and `alt` describe the generated asset. Inline styles specify its background, text and layout; this is not a regular HTML page.

### src/app/robots.ts

Generates `/robots.txt`, instructions for web crawlers.

Preview deployments or an explicit no-index setting disallow crawling the whole site. Otherwise public pages are allowed and `/api/` is disallowed. The sitemap URL is built from the canonical brand site URL.

### src/app/sitemap.ts

Generates `/sitemap.xml` for public routes.

It maps an explicit route list to absolute URLs based on the canonical site URL. Each entry includes lastModified, changeFrequency and priority hints. It is a crawler aid, not a guarantee of indexing.

### src/app/api/lead/route.ts

This is a Node.js server route that handles POST requests from the contact form. It is not browser UI.

It verifies JSON content type and same origin; checks Content-Length and streams the body with a 10,000-byte limit; parses JSON; handles a honeypot; validates via the shared Zod schema; requires the legal and provider gates; extracts and hashes the IP; applies a sliding-window Upstash rate limit; and sends a plain-text email through Resend. Responses use appropriate HTTP statuses such as 400, 403, 413, 429, 502 and 503. Provider failures are logged without logging form content.

```
export const runtime = "nodejs";
const maximumBodyBytes = 10_000;

export async function POST(request: Request) {
  // Check request format, origin, size, honeypot and schema.
  // Require legal approvals and private provider credentials.
  // Rate-limit, send email, then return JSON status.
}
```

> Why each protection is separate
> UI validation helps honest visitors, but the server is the security boundary. Browser code can be changed, so the API checks all important inputs again.

# Part II - Reusable components, file by file

A component is a reusable function that describes interface elements. A client component is marked `use client` when it needs browser state, event handlers or browser APIs.

### src/components/Header.tsx

Shared responsive header/navigation used by the root layout.

It uses `useState` for mobile menu open/closed, a ref to the menu button for keyboard focus, and an effect that subscribes to Escape key events. Cleanup removes the event listener. `closeMenu` is shared by navigation links. Product links are built from content data inside native details/summary. The component tracks navigation/CTA events through the analytics wrapper. aria-expanded and aria-controls expose menu state and relationship.

```
const [menuOpen, setMenuOpen] = useState(false);
const menuButton = useRef<HTMLButtonElement>(null);
// Effect listens for Escape and restores focus.
<button aria-expanded={menuOpen} onClick={() => setMenuOpen((open) => !open)} />
```

### src/components/AnnouncementBar.tsx

A dismissible Medicoplasma launch announcement.

The component uses `useSyncExternalStore` to subscribe to a custom browser event and read the dismissal value from sessionStorage. Dismissing stores a true string for the browser tab session and dispatches that event so React updates. Server snapshot returns false, avoiding a browser API call during server rendering. No personal data is stored.

### src/components/Footer.tsx

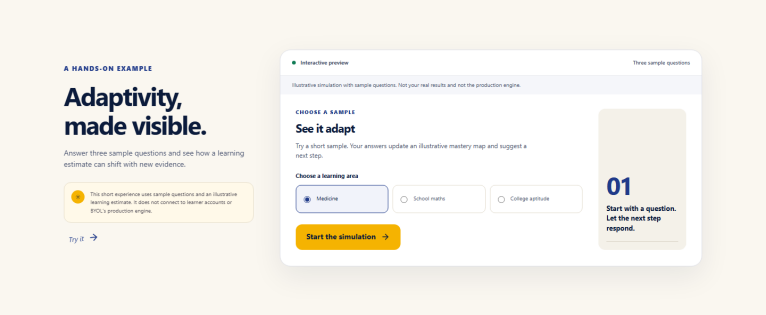
Shared footer links for platform destinations, company pages and legal routes.

It reuses brand/copy/platform data, maps product links from the platform array, uses Next Link for internal pages, and marks external links with a small external icon.

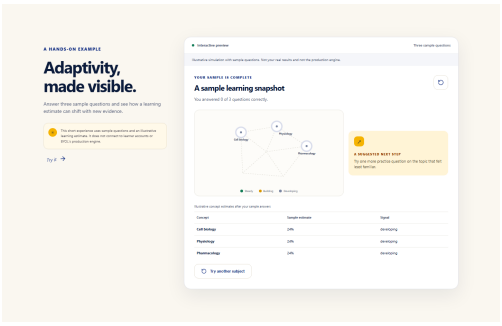
## src/components/AdaptiveDemo.tsx

The interactive local sample: subject choice, three questions, feedback, concept estimates and result screen.

A union type constrains its four UI states. Several state variables track subject, index, selected answer, estimates and correct count. Refs let useEffect move keyboard focus when the state changes. startDemo resets state; submitAnswer compares the chosen index with the answer index, calculates an illustrative estimate and shows feedback; moveForward advances or finishes; reset returns to idle. `useMemo` derives concept names from completed questions. The result contains both MasteryMap and a semantic table. Analytics calls are gated.




*Starting state for the interactive sample.*




*Result state displays the sample map and text table.*

## src/components/MasteryMap.tsx

Reusable SVG concept map used on the homepage and demo results.

Props optionally supply estimates, concepts and compact styling; defaults produce an illustrative map. `useId()` creates unique SVG title/description references. Node positions are coordinates in an SVG viewBox. Each estimate becomes a signal and CSS class, which controls color. The component also emits a screen-reader-only HTML table with the same data, so a diagram is not the only way to understand the values.

## src/components/LeadForm.tsx

Client-side contact form and the visitor experience around submission.

The Intent union limits audience values. State stores intent, field errors, status and submitting state. A ref makes form-start analytics fire once. The public enable flag decides whether to show the fields or a disabled explanation. On submission it creates FormData, changes checkbox consent into a boolean, sends JSON to `/api/lead`, reads field/server errors, announces status and resets on success. Institution intent conditionally renders organization input. Labels, aria-invalid/describedby and live status improve accessibility.

## src/components/AnalyticsEvents.tsx

Optional delegated click and scroll-depth listeners.

A useEffect returns immediately unless analytics is enabled. A single document click listener finds the nearest element with `data-analytics-event`; values become typed event properties. A passive scroll listener reports 25/50/75/100 percent only once each per page visit. The cleanup function removes listeners. The component returns null because it adds behavior, not visible UI.

## src/components/ApprovalNotice.tsx

Shows a small “Needs approval” label only in development for an unapproved item.

It returns null for approved items and for every production build. This gives editors a preview warning without accidentally publishing that warning or unapproved entries as normal content.

## src/components/JsonLd.tsx

Embeds JSON-LD structured data for search engines.

The component serializes a typed object to JSON and replaces `<` with its escaped Unicode representation before assigning it to a script element. This helps prevent user-controlled text from closing the script tag. `dangerouslySetInnerHTML` is limited to this

carefully serialized data.

```
const serialized = JSON.stringify(data).replace(/</g, "\\u003c");
<script type="application/ld+json"
  dangerouslySetInnerHTML={{ __html: serialized }} />
```

# Part III - Content and reusable logic

Content files describe what appears. Library files describe how repeated calculations, validation, analytics and search data work.

## src/content/site.ts

The central typed source for brand settings, navigation, page copy, product cards, audience links, benefits, conceptual engine steps, principles, proof figures and demo labels.

Types such as Approvable<T> and Platform describe the required shape. `as const` preserves literal values. `brand.siteUrl` reads the optional canonical URL with a fallback. Arrays hold repeated records, so the same platforms are reused by homepage/header/footer/contact. `shippable()` filters unapproved items in production while retaining drafts for preview in development.

```
export type Approvable<T> = T & { approved: boolean; note?: string };
export type Platform = Approvable<{
  id: "medicoplasma" | "schools" | "colleges";
  name: string;
  href: string;
}>;

export function shippable<T extends { approved: boolean }>(items: readonly T[]) {
  return process.env.NODE_ENV === "production"
    ? items.filter((item) => item.approved)
    : [...items];
}
```

## src/content/demo-data.ts

Question bank for the illustrative demo.

DemoQuestion describes concept, prompt, readonly options, answer index and explanation. `demoQuestions` is a record keyed by subject. Each subject has three sample questions. The `answer` is an array index starting at zero; the demo and tests rely on this convention.

## src/lib/adaptive/mastery.ts

Pure deterministic functions for the illustrative probability update and readable signal categories.

BktParameters types guess/slip/learn. The defaults are guess 0.2, slip 0.1, learn 0.15. updateMastery rejects invalid probabilities, computes likelihoods for correct/incorrect evidence, calculates a posterior safely, then adds learning chance. masterySignal validates its input and returns developing/building/steady at fixed thresholds. Because this is a pure helper, unit tests can call it without a browser.

## src/lib/validation/lead.ts

One Zod schema used to validate enquiry data on the server.

String trimming and length limits constrain input. Email uses Zod's email rule. Intent is a finite enum. Consent must be the literal boolean true. refine adds a rule that institution enquiries need an organization. `z.infer` derives LeadInput from the schema, avoiding a separately maintained duplicate TypeScript shape.

```
export const leadSchema = z.object({
  name: z.string().trim().min(2).max(100),
  email: z.string().trim().email().max(200),
  intent: z.enum(["learner", "parent", "institution", "other"]),
  consent: z.literal(true),
}).refine(institutionHasOrganisation);
export type LeadInput = z.infer<typeof leadSchema>;
```

## src/lib/analytics.ts

A single wrapper around the analytics vendor's track function.

Event properties are typed as strings/numbers/booleans. The wrapper sends only in production and only when the public analytics flag is exactly true. Call sites therefore use one policy gate instead of importing the vendor everywhere.

## src/lib/seo.ts

Small helpers/data for absolute URLs and Organization/WebSite JSON-LD.

`absoluteUrl(path)` resolves a path against the brand site URL. The exported structured-data objects use Schema.org context and the site name, URL, description and locale. Homepage passes these objects to JsonLd.

### src/styles/globals.css

The shared design system and all global styles.

The root block defines color, font, radius and shadow variables. Global rules normalize box sizing, type and focus behavior. Named classes style the header, hero, cards, demo, forms, footer and page sections. Three media breakpoints adapt the layouts to smaller viewports. The reduced-motion media query removes nonessential transitions. CSS is imported once by layout.tsx.

```
:root {
  --color-ink: #0b1b3b;
  --color-canvas: #faf7f0;
  --color-primary: #1f3a8a;
}
:focus-visible { outline: 3px solid var(--color-primary); }
@media (max-width: 600px) { /* phone layout adjustments */ }
@media (prefers-reduced-motion: reduce) { /* less motion */ }
```

# Part IV - Tests, one file at a time

Tests describe behavior the project must keep. Vitest runs the smaller unit tests; Playwright opens actual browser pages for end-to-end behavior.

### tests/unit/mastery.test.ts

Checks updateMastery and masterySignal.

It verifies correct answers increase and incorrect answers decrease estimates, outputs stay between 0 and 1 across boundary priors, invalid parameter values throw RangeError, and signal thresholds return the expected labels. This gives confidence in the illustrative calculation.

### tests/unit/content.test.ts

Checks approval handling and the shape of question data.

Vitest temporarily stubs NODE_ENV to verify approved-only production filtering and development draft visibility, then restores environment values after each test. It checks current platform/stat entries are approved and each sample question has a valid prompt, concept, options and answer index.

### tests/unit/lead-validation.test.ts

Exercises the leadSchema directly.

A valid learner lead passes. Institution leads without an organization fail. Missing consent, malformed email and oversized messages fail. These tests lock in the validation contract without sending any email.

### tests/unit/lead-route.test.ts

Tests important API rejection paths without calling real providers.

makeRequest builds a Request with controllable origin/content type. The tests assert cross-origin requests get 403, non-JSON gets 415, oversized body gets 413, honeypot receives a harmless success-shaped response without email, and a valid payload is still blocked with 503 until service/legal settings exist.

### tests/e2e/site.spec.ts

Playwright browser tests for routes, accessibility, demo flow and mobile navigation.

The page list is run at phone and desktop projects. Axe checks WCAG-tagged rules and asserts no serious/critical violations. The demo test clicks through all questions, confirms feedback and the text table/disclaimer, then runs axe on results. Another test opens the mobile menu. Additional tests check horizontal overflow and branded 404 behavior.

# Part V - Root configuration and documentation files

These files sit beside src/ and tests/. They configure Node/npm, Next.js, TypeScript, linters and test runners, or document how people should work with the website.

| File | What it controls | What a beginner should know |
|---|---|---|
| package.json | Project identity, dependencies and npm commands. | Use scripts; do not manually run package internals. |
| package-lock.json | Exact resolved dependency tree. | npm ci/install maintains it; avoid hand editing. |
| tsconfig.json | TypeScript compiler rules and aliases. | strict enables useful checks; @/* maps to src/*. |

| File | What it controls | What a beginner should know |
|---|---|---|
| next.config.ts | Next.js configuration and HTTP response headers. | Headers are applied to every route. |
| eslint.config.mjs | Lint rules for Next core web vitals and TypeScript. | npm run lint runs these rules. |
| vitest.config.ts | Unit-test plugin, path alias and test file pattern. | Vitest runs tests/unit. |
| playwright.config.ts | Browser projects, local test server and report/trace settings. | It tests phone and desktop widths. |
| next-env.d.ts | Generated Next.js type references. | Do not edit by hand. |
| .gitignore | Files Git should ignore. | Includes secrets, caches and test output. |
| .env.example | Template of setting names/defaults. | Contains no actual credentials. |
| README.md | Local setup and deployment variable instructions. | Start here to run the site. |
| LAUNCH_CHECKLIST.md | Requirements R1-R15 and remaining approvals. | Some items are deliberately blocked/partial. |
| AGENTS.md | Coding-agent workspace instructions. | Not served to site visitors. |
| .github/workflows/ci.yml | Automated checks for pull requests and main pushes. | Explained in detail below. |

## package.json

The npm project manifest: project metadata, dependency versions and named commands.

Dependencies include Next, React, Zod, Resend, Upstash and analytics. Development dependencies include TypeScript, ESLint, Vitest and Playwright. Versions are pinned to exact versions. Scripts map simple names like `npm run dev` to the underlying tools.

```
"scripts": {
  "dev": "next dev",
  "build": "next build",
  "start": "next start",
  "lint": "eslint .",
  "typecheck": "tsc --noEmit",
  "test": "vitest run",
  "test:e2e": "playwright test"
}
```

## package-lock.json

npm's lockfile for the complete dependency tree.

It records precise direct and nested package versions, integrity details and dependency relationships. This helps another machine install the same tree. Change dependencies with npm commands so package.json and the lockfile stay aligned.

## tsconfig.json

TypeScript compiler configuration.

Strict mode and noUncheckedIndexedAccess make uncertain/missing values visible to the type checker. `noEmit` prevents TypeScript writing JavaScript output because Next handles the build. The `@/*` path maps imports to src/*. JSX uses React's modern transform. Generated Next route types are included.

## next.config.ts

Next.js settings for response behavior and security headers.

poweredByHeader false removes a framework-identifying header. The headers function applies HSTS, nosniff, referrer policy, permissions policy and frame denial to all paths. HSTS should be used in a deployment that correctly serves HTTPS.

## eslint.config.mjs

ESLint flat configuration composed from Next core-web-vitals and TypeScript presets.

It globally ignores generated folders and next-env.d.ts. ESLint finds patterns that are likely mistakes or violate project/framework conventions; it does not prove that the product behavior is correct.

## vitest.config.ts

Configures the unit-test runner.

The React plugin supports TSX transforms. The alias matches the app's @ import shortcut. The node environment is used for tests matching tests/**/*.test.ts.

## playwright.config.ts

Configures browser automation.

Tests run in mobile Chromium and desktop Chromium projects. Playwright starts Next dev on port 3200, uses that base URL, keeps traces on failures, and uses CI-specific retry/reporter behavior. Chromium emulates the selected viewport/device settings.

# Part VI - CI workflow and operations

## .github/workflows/ci.yml

GitHub Actions automation named `ci`, triggered for pull requests and pushes to main.

A clean Ubuntu runner checks out the source, installs Node 22 with npm cache, runs npm ci from the lockfile, then lint, typecheck, unit tests and production build. It installs Chromium dependencies, runs browser tests, and ends with npm audit at high severity. This workflow verifies changes; it does not deploy or supply approvals.

```
name: ci
on:
  pull_request:
  push:
    branches: [main]
jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
      - run: npm ci
      - run: npm run lint
      - run: npm run typecheck
      - run: npm test
      - run: npm run build
      - run: npx playwright install --with-deps chromium
      - run: npm run test:e2e
      - run: npm audit --audit-level=high
```

## .env.example

Safe example names for runtime configuration.

It includes public site URL, disabled-by-default analytics/form flags, server legal gate, blank sender/recipient/provider credentials, and Upstash settings. Copy values into the hosting dashboard or a local ignored .env file; never put live secrets in source control.

## .gitignore

Tells Git which generated, local or sensitive files should not be tracked.

It ignores node_modules, .next, build output, coverage, Playwright artifacts, `.env*` and TypeScript build info, while explicitly allowing .env.example. This prevents large generated folders and local secret files from entering commits.

## README.md

Project starting instructions.

Lists Node/npm prerequisites, local `npm install` and `npm run dev`, verification commands, environment variables and important legal/provider gates. It links to the launch checklist.

## LAUNCH_CHECKLIST.md

Requirement-by-requirement delivery record and launch blockers.

R1-R15 describe demo, routing, CTAs, proof, responsible AI, legal behavior, mobile/performance, accessibility, SEO, analytics, lead capture, governance, testing/CI and localization readiness. Status distinguishes done, partial and blocked. The required launch section makes human approvals explicit.

## AGENTS.md

Workspace-specific instructions for coding agents, not website runtime.

It asks agents to consult documentation bundled with this installed Next version before making Next API changes and notes the framework may regenerate the file. It does not affect the public website.

## next-env.d.ts and tsconfig.tsbuildinfo

Generated TypeScript/Next support files.

next-env.d.ts references framework types and generated route types; its own comment says not to edit it. tsconfig.tsbuildinfo is an incremental compilation cache ignored by Git. Both can be regenerated by project tools.

# Final reference - complete authored file inventory

This checklist covers the authored application and test files present in the project when this guide was prepared.

| Area | Files covered |
|---|---|
| App routes | layout.tsx, page.tsx, about/page.tsx, blog/page.tsx, contact/page.tsx, privacy/page.tsx, terms/page.tsx, styleguide/page.tsx. |
| App system files | error.tsx, not-found.tsx, opengraph-image.tsx, robots.ts, sitemap.ts, api/lead/route.ts. |
| Components | AdaptiveDemo.tsx, AnalyticsEvents.tsx, AnnouncementBar.tsx, ApprovalNotice.tsx, Footer.tsx, Header.tsx, JsonLd.tsx, LeadForm.tsx, MasteryMap.tsx. |
| Content | demo-data.ts, site.ts. |
| Library helpers | analytics.ts, seo.ts, adaptive/mastery.ts, validation/lead.ts. |
| Styles | globals.css. |
| Unit tests | content.test.ts, lead-route.test.ts, lead-validation.test.ts, mastery.test.ts. |
| Browser test | tests/e2e/site.spec.ts. |
| Root/config/CI | package.json, package-lock.json, tsconfig.json, next.config.ts, eslint.config.mjs, vitest.config.ts, playwright.config.ts, next-env.d.ts, .env.example, .gitignore, .github/workflows/ci.yml. |
| Docs/references | README.md, LAUNCH_CHECKLIST.md, AGENTS.md, supplied build/research reference documents, previous beginner/code PDFs. |

## Choose a file by the change you want

| Goal | Start at |
|---|---|
| Change homepage words | src/content/site.ts |
| Reorder homepage sections | src/app/page.tsx |
| Change a sample question | src/content/demo-data.ts |
| Adjust sample estimate formula | src/lib/adaptive/mastery.ts and its unit test |
| Change menu behavior | src/components/Header.tsx |
| Change contact UI | src/components/LeadForm.tsx |
| Change server form protections | src/app/api/lead/route.ts and tests/unit/lead-route.test.ts |
| Change colors/mobile layout | src/styles/globals.css |
| Change automated checks | tests/ and the relevant runner config |
| Understand launch readiness | LAUNCH_CHECKLIST.md |

## A safe workflow for every change

```
1. Open the file with Ctrl+P in VS Code.
2. Read the whole nearby function/data object before editing.
3. Make one small related change.
4. Save and inspect the local website.
5. Run lint, typecheck and relevant tests.
6. Review the browser and terminal results.
7. Get approval for business, legal or factual claims.
```

Two important launch boundaries

The adaptive demo is illustrative, not the deployed learning engine. The contact form is intentionally disabled until legal copy/consent approval and secure email/rate-limit configuration are complete.

End of file-by-file explanation. For screenshots of the complete website pages, see BYOL_Academy_Beginner_Guide.pdf.